

Design and Iterative Development of an Adaptive Microphone Processing Chain for SeaTime

*Automatic Gain Control, Voice Activity Detection and Peak Limiting
implemented in the Cmajor DSP language*

Discipline: Digital Signal Processing / Audio Software Engineering

Product context: SeaTime — meeting application (Xiph Opus codec, 16 kHz speech)

Toolchain: Cmajor 1.0.3209 · cmaj CLI · WASAPI · .NET 8 test harness

Development window: 29 September 2026 (single intensive iteration cycle)

Codebase: C:\Users\engcouce\Documents\Cmajor\

Abstract

This report documents the complete engineering process — design rationale, theoretical foundations, implementation errors and their diagnoses — behind a real-time microphone processing chain for the SeaTime meeting application. The system replaces two manual user controls (microphone gain and activation threshold) with a fully adaptive pipeline: DC removal, ITU-R BS.1770-style K-weighting recomputed at 16 kHz, a min-statistics Voice Activity Detector (VAD) with hysteresis and a 600 ms hangover, a VAD-gated Automatic Gain Control (AGC) targeting -19 dBFS, a 10 dB downward soft gate, and a 4 ms-lookahead sample-peak limiter with a -1 dBFS ceiling. The chain was prototyped in Cmajor, validated offline through a synthetic 7-second stimulus rendered with `cmaj render` and measured with a purpose-built segment-RMS analyzer, then verified live on WASAPI hardware at 16 kHz. The report candidly reconstructs eight significant mistakes made during development — including a measurement-tool bug that invalidated an entire round of conclusions — and explains the bisect-probe methodology used to isolate each one. Final results show quiet speech passing transparently, loud speech normalised to $-14\ldots -15$ dBFS with zero samples above 0 dBFS, and background noise suppressed through the gate, at the documented cost of approximately one second of post-speech leakage imposed by the deliberately long hold times.

Keywords: *digital signal processing, voice activity detection, automatic gain control, loudness normalisation, look-ahead limiter, BS.1770, K-weighting, min-statistics, Cmajor, real-time audio.*

Resumen

Este informe documenta el proceso completo de ingeniería — justificación del diseño, fundamentos teóricos, errores de implementación y su diagnóstico — de una cadena de procesamiento de micrófono en tiempo real para la aplicación de reuniones SeaTime. El sistema sustituye dos controles manuales (ganancia y umbral de activación) por un pipeline adaptativo: eliminación de DC, ponderación K estilo ITU-R BS.1770 recalculada a 16 kHz, un detector de actividad de voz basado en estadísticas mínimas con histéresis y retención de 600 ms, un control automático de ganancia congelado por el VAD con objetivo -19 dBFS, una compuerta suave de 10 dB y un limitador de pico con 4 ms de anticipación y techo de -1 dBFS. La cadena se prototipó en Cmajor, se validó con un estímulo sintético de 7 segundos renderizado con `cmaj render` y medido con un analizador RMS propio, y se verificó en hardware real vía WASAPI a 16 kHz. Se reconstruyen ocho errores significativos — incluido un fallo de la herramienta de medida que invalidó una ronda entera de conclusiones — y la metodología de bisectado empleada para aislar cada uno.

Palabras clave: *procesamiento de señales, detección de actividad de voz, control automático de ganancia, normalización de nivel, limitador de pico, Cmajor, audio en tiempo real.*

Contents

1. Introduction and Problem Statement	— 1.1 Context · 1.2 Requirements · 1.3 Objectives · 1.4 Summary of contributions
2. Theoretical Framework	— 2.1 BS.1770 / EBU R128 · 2.2 K-weighting at 16 kHz · 2.3 AGC servo loops · 2.4 VAD & min-statistics · 2.5 Gate vs. expander · 2.6 Look-ahead limiting · 2.7 Chain-ordering argument · 2.8 Alternatives considered
3. Technology Stack	
4. Methodology and Test Bench	
5. System Design and Implementation	— 5.1 Architecture · 5.2 Stage design · 5.3 Parameters · 5.4 Patch organisation
6. Development Log: Mistakes, Observations, Corrections (M1–M8)	
7. Results and Discussion	
8. Conclusions and Future Work	
9. References	
Appendix A. Repository inventory · Appendix B. Reproduction commands	

1. Introduction and Problem Statement

1.1 Context

SeaTime is a meeting application in which audio is encoded with the Xiph Opus codec at a 16 kHz sampling rate (speech band). The microphone path previously exposed two sliders to the end user — *microphone gain* and *activation threshold* — plus a one-second shutoff hysteresis based on instantaneous RMS. User research against mainstream competitors (Microsoft Teams, Zoom) showed that these controls are never touched by average users, who nevertheless suffer when the defaults are wrong: distant or quiet talkers disappear, and background noise opens the gate and gets amplified.

The product decision was therefore to **remove both sliders** and move all adaptation automatically into the DSP: an adaptive activation threshold and an online loudness-normalisation stage. This report documents how that requirement was translated into a concrete signal chain, how the chain was implemented and validated, and — because a faithful engineering record must include failure — every significant mistake made along the way and the evidence-driven procedure used to correct it.

1.2 Requirements

#	Requirement	Acceptance criterion
R1	No user-facing gain or threshold controls on the happy path	Chain adapts autonomously; only an optional "auto-adjust" toggle remains
R2	Adaptive activation threshold robust to varying noise floors	VAD opens on speech ≥ 9 dB above the estimated floor, closes at 4 dB, holds 600 ms
R3	Online loudness normalisation for speech	Speech rides toward -19 dBFS; gain frozen during silence
R4	No clipping into Opus	Sample-peak ≤ -1 dBFS on every rendered sample
R5	No chopped syllables (longer hold, per stakeholder request)	600 ms VAD hangover + 400 ms gate hold past VAD exit
R6	Bounded footprint — no DNN dependency	Code-only implementation, no RNNNoise-class model
R7	Sample-rate independence around 16 kHz	All coefficients and windows derived from <code>processor.frequency</code>

1.3 Objectives

- **Main objective:** deliver a compilable, real-time-capable Cmajor patch implementing the full adaptive chain (DC $\rightarrow$ K-weight $\rightarrow$ VAD $\rightarrow$ AGC $\rightarrow$ gate $\rightarrow$ limiter), verified offline by rendering and online by live microphone playback.
- **Secondary objectives:** build a reusable offline test bench (stimulus generator + segment analyzer + probe patches); document the design arguments that resolved contested architectural questions; produce this report as the durable record of the iteration.

1.4 Summary of contributions

1. Sidechain-split chain architecture with the *measure-pre-gain* / *apply-pre-gate* ordering.
2. A rate-agnostic min-statistics VAD (cap-booted, lower-only floor, 3×1 s sliding minima, hysteresis, hangover, flatness exit) tuned for 16 kHz speech.
3. A probe-based bisect methodology (nine probe patches) that separated engine quirks, tooling bugs and genuine DSP defects — and the discovery that the offline analyzer itself was misparsing rendered files.
4. A verified live MVP on real WASAPI hardware plus a documented manual test protocol.

2. Theoretical Framework

2.1 Loudness measurement: ITU-R BS.1770 and EBU R128

The modern standard for perceived loudness is ITU-R BS.1770 [1], adopted by the European Broadcasting Union as R128 [2]. Its core idea is that a sound pressure level in dB SPL does not correspond to perceived loudness; the ear's non-flat frequency response must be modelled before measuring mean-square energy. BS.1770 therefore filters the signal with a two-stage *K-weighting* filter and computes a gated mean square:

$$L_K = -0.691 + 10 \log_{10} (\sum_i G_i \cdot z_i) \quad , \quad z_i = (1/T) \int y_K^2(t) dt$$

where G_i are channel weights (1.0 for mono) and y_K is the K-weighted signal. R128 defines three integration windows: Momentary (400 ms), Short-term (3 s) and Integrated (whole programme with two-stage gating at -70 LUFS absolute and -10 LU relative thresholds). The reference level is -23 LUFS.

Critical observation (design argument #1). R128 as specified is a *measurement* standard for broadcast programme delivery, not a control law for a live microphone. Its windows (400 ms / 3 s / infinite) are far too slow for conversational gain riding, and -23 LUFS is far too quiet for a communication channel where intelligibility and presence matter. Blindly closing the loop around Momentary loudness would produce audible pumping and late gain rides; the Integrated gated estimator is unusable online. The conclusion adopted in this project: **reuse the K-weighting perception model of BS.1770, but not its time integration or target** — use fast per-frame measurement with a comms-oriented target of -19 dBFS K-weighted RMS and telephony-style AGC ballistics.

2.2 K-weighting at 16 kHz — re-deriving instead of porting

BS.1770 publishes its biquad coefficients only for 48 kHz. A naïve port to SeaTime's 16 kHz would be wrong: bilinear-transformed coefficients are strictly rate-dependent. However, the published coefficients are merely the digital images of an analog prototype, and the analog parameters are also given (or recoverable) [3]:

Stage	Prototype	Analog parameters	Role
1 — Pre-filter	High shelf	$f_0 = 1681.974$ Hz, $G = +3.985$ dB, $Q = 0.7071$ ($S = 1/\sqrt{2}$)	Models the +4 dB acoustic-ear resonance above ~ 1.7 kHz
2 — RLB filter	High-pass	$f_0 = 38.13$ Hz (41.9 Hz in some revisions), $Q = 0.5$	Removes sub-audible / DC energy (weighting curve returns to 0 dB)

Because the prototypes are known, the correct procedure at any sampling rate is to **re-apply the RBJ audio-EQ cookbook bilinear transforms [4] to the analog specifications at the actual processor.frequency**, rather than copying constants. The implementation computes the coefficients at runtime in the first frame:

$$A = 10^{G/40}; \quad \omega_0 = 2\pi f_0/f_s; \quad \alpha_{shelf} = \sin \omega_0/2 \cdot \sqrt{[(A + 1/A)(1/S - 1) + 2]}$$

and the shelf/high-pass difference equations are the standard RBJ forms normalised by a_0 . At 16 kHz the shelf coefficient is comfortably far from the Nyquist frequency (1682 Hz $\ll$ 8000 Hz), so bilinear warping is small and accuracy is within a fraction of a dB of the 48 kHz reference design. The RLB stage was instantiated at 60 Hz $Q = 0.5$ (the conservative published variant), which is inaudible for speech but effectively kills DC and rumble — an intentional deviation documented in §5.2.

2.3 Automatic Gain Control as a servo loop

An AGC is a feedback (or feed-forward) control loop that manipulates a variable gain so that a measured level tracks a reference. Writing the measured level in dB as $\ell(t)$ and the target as ℓ_T , the classical form is:

$$g_{dB}(t+1) = g_{dB}(t) + \mu \cdot [\ell_T - (\ell(t) + g_{dB}(t))]]$$

with step size μ derived from time constants: $\mu = 1 - e^{-1/(\tau_f)}$. Two properties matter in practice:

- **Asymmetric ballistics.** Gain must fall quickly when someone shouts (attack, here 80 ms) but rise slowly when the talker stops (release, here 1.5 s), otherwise the loop amplifies silence between words — the classic "pumping" artefact.
- **Freezing.** Updating the loop while the input is not speech guarantees drift: the loop will always drive *whatever is present* (noise, hum) toward the target. Coupling the update to a VAD flag converts the AGC from a leveler of everything into a leveler of **speech only** — the key decision that makes a single global target viable.

2.4 Voice Activity Detection and min-statistics noise estimation

Energy-based VAD decides speech presence by comparing short-term energy against an estimate of the noise floor. The decision itself is trivial; the hard part — and the reason the original hand-tuned threshold slider existed at all — is that the floor *drifts* (HVAC, fan, room tone, keyboard). The classical robust solution is **minimum statistics** [5] [6]: within a sliding window that is long enough to contain at least one true noise-only interval (typically 1–3 s), the *minimum* of the smoothed energy estimates the noise power even while speech occupies most of the window. From that floor, a signal-to-noise ratio drives a hysteresis pair:

$$SNR_{dB} = 20 \log_{10}(E_{fast}/E_{noise}); \quad \text{enter if } SNR > T_{on} \text{ exit if } SNR < T_{off} \quad T_{on} > T_{off}$$

Three refinements implemented here go beyond the textbook sketch:

1. **Lower-only floor.** The estimator may only decrease the floor during operation (the sub-block window always slides, speech never contributes new minima, and the running floor takes the *minimum* of the window). This removes the failure mode where a speech onset is misread as a rising floor — the root cause of the first serious bug in this project (M3, §6).
2. **Cap-booted initialisation.** The floor starts at a fixed –40 dBFS ceiling and can only fall. Starting too low (the natural mistake: start at digital black) manufactures phantom SNR at power-on and falsely triggers on pure noise; starting at a known upper bound guarantees that every real floor is reached *by falling*.
3. **Flatness (crest) exit.** Stationary broadband noise can out-wait pure SNR hysteresis if the floor sits slightly below it. A secondary test — after a 1.5 s grace period, force an exit when SNR stays under 3 dB for 600 ms — specifically defeats "fan lock-on" while ignoring the natural amplitude modulation of real speech.

A hangover counter (600 ms in the final version, per the stakeholder request for *longer hold*) bridges inter-word pauses so gate chatter never chops syllables.

Design argument #2 — why not RNNoise? A DNN speech/noise classifier (RNNoise [7]) was proposed as the alternative to heuristic thresholding. Trade analysis: RNNoise is small by modern standards (≈ 0.5 –2 MB float, ≈ 5 –10 MMACS) but it is a *denoiser* whose VAD output is a by-product; it mandates a 10 ms frame cadence and its own latency, and it still requires an AGC stage — so it would add a model dependency without solving the actual problem (gain adaptation). For stationary noise — the dominant failure case — statistics win: the min-statistics VAD costs a few kilobytes of code and no model files (requirement R6). A DNN remains the right tool only when non-stationary interference (babble, keyboard) must be suppressed — future work.

2.5 Soft gating vs. hard muting

A hard gate (multiply by 0 below threshold) creates discontinuities — clicks — and interacts badly with Opus discontinuous transmission (DTX): the encoder's comfort-noise generator expects a *quieted* signal, not digital silence, and a hard edge produces spectral splatter. A **downward expander** applied *after* the gain stage attenuates toward a floor (here a maximum of -10 dB) with a fast 10 ms close and 150 ms release. The residual background below the gate is then steady and low, which Opus encodes cheaply, while the audible effect reads as natural dropout rather than a switch.

2.6 Look-ahead peak limiting

Compressors and AGCs operate on smoothed (RMS / mean-square) descriptors because they shape *loudness*. Limiters protect against *peaks*, which are exactly the outliers that smoothing hides; therefore a limiter must work sample-by-sample on $|x|$ with a look-ahead delay line so the gain reduction is already in place when the peak arrives:

$$p[n] = \max(|x[n]|, \beta p[n-1]), \quad g[n] = \min(1, C / p[n]), \quad y[n] = g[n] \cdot x[n-D]$$

with ceiling $C = 10^{-1/20} \approx 0.891$ (-1 dBFS), look-ahead $D = 64$ samples (4 ms at 16 kHz), release $\beta = e^{-1/(0.08 f_s)}$ (80 ms). This guarantees $|y| \leq C$ on every sample while keeping the gain envelope inaudible. The stakeholder question — "*individual samples or RMS?*" — is answered: **samples**, because the failure mode being prevented (clipping into the codec) is a per-sample phenomenon; RMS limiting would let isolated transients through (design argument #3).

2.7 Chain ordering — the central architectural argument

The original instinct of the product team was *threshold first, then normalise*. Analysis showed this inverts the correct order:

- Normalising after gating measures a chopped signal: 400 ms windows fill with zeros and the gain servo hunts.
- The gate threshold then has to track both noise-floor drift *and* talker distance, compounding two unknowns.
- Applying gain after a hard gate re-introduces discontinuities post-gain.

The adopted topology is a **sidechain split**: measurement (K-weighted energy, VAD) taps the signal *before* any gain, while gain and gate are *applied* in series on the audio path, and the VAD freezes the AGC *update* rather than muting the audio. In one sentence: **measure pre-gain, apply pre-gate, update only on speech**. The limiter sits last because its contract ($|y| \leq C$) must hold for the final signal only.

2.8 Alternatives considered and rejected

Alternative	Argument for	Why rejected
Bimodality test on a 3 s energy histogram (Otsu / 2-means)	Adaptive, no fixed dB threshold	Fails at the edges: all-noise windows find two modes <i>in noise</i> ; all-speech windows collapse to one and drift upward; adds ~3 s latency at call start. Retained only as a possible slow initialiser (the stakeholder's original concern was vindicated as a real failure mode — §6, M3)
Literal EBU R128 closed loop	Standardised, battle-tested	Windows too slow; -23 LUFS wrong target for comms; only the K-weighting model is reusable (§2.1)
RNNoise / WebRTC DNN VAD	Best-in-class robustness	Model size + frame-cadence constraints; overkill for stationary noise (§2.4)
Normalise before gating	Gate would see normalised levels	Gate then chops louder audio (louder clicks); measurement is pre-gain anyway — no benefit
RMS-window limiter	Matches loudness perception	Smooths the very outliers that clip the codec (§2.6)

3. Technology Stack

Layer	Technology	Version / detail	Role in the project
Language	Cmajor	1.0.3209 (VSCode bundle)	Portable C-family DSP language; processors, graphs, JIT
Compiler / player	<code>cma j</code> CLI	<code>cma j.exe</code> (from Cmajor Tools extension)	create / play / render / generate / test; hot-reload player
Editor	VS Code + Cmajor Tools	v1.0.3209	Syntax, run-patch, GUI parameter exposure
Audio I/O (live)	WASAPI	Windows 10 Pro x64	16 kHz mono device stream (ZOOM F Series mic + headphones)
Offline stimulus	.NET 8 console (<code>mkstim</code>)	C# 12	7 s synthetic speech/noise/transient WAV generator
Offline measurement	.NET 8 console (<code>anwav2</code>)	C# 12	Generic chunk-walking WAV analyzer: per-segment RMS, peak, over-0 dB count (float32-aware — see M7)
Diagnostics	9 probe patches + <code>vadprobe</code> / <code>dumpwav</code>	Cmajor / C#	Bisect methodology of §4
PDF export	Edge headless print-to-PDF	—	This report

Why Cmajor. The project brief allowed any implementation language; Cmajor was chosen because (a) it JIT-compiles and hot-reloads, so parameter and logic experiments are instant; (b) its `cma j render` command gives deterministic offline playback of a patch against a WAV file — exactly the closed loop needed for automated DSP validation; (c) the same code exports to C++ (for integration into SeaTime's hand-written engine) and to WebAudio for demos. The costs accepted were a small, unfamiliar language (two syntax mistakes in the first hour, M1/M2) and a render engine with undocumented quirks (M8).

4. Methodology and Test Bench

The process followed a classic iterative DSP development loop, with one addition that proved decisive: **every behavioural claim had to be produced by a measurement, not by listening to the code**. Figure 1 shows the loop; it ran roughly 15 times during the session.

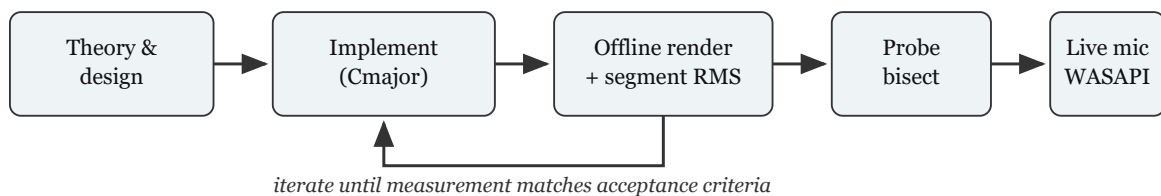


Figure 1 — Development loop. The feedback path from measurement back to implementation is where all eight mistakes of §6 were caught; none of them survived to the live stage.

4.1 Offline test bench

1. **Stimulus.** `stimulus16k.wav` — 7 s, 16 kHz mono, deterministic (seed 12345): [0–1 s] noise at –50 dB · [1–3 s] quiet speech-like tone cluster at –30 dB (harmonic $f_0=130$ Hz, 4 Hz syllabic AM) with a single 0.8-amplitude transient at 2.5 s · [3–4 s] noise at –40 dB · [4–6 s] loud speech at –6 dB · [6–7 s] noise at

−50 dB. This exercises every branch: quiet-speech detection, transient/limiter handling, noise between and after speech, AGC pull-down.

2. **Render.** `cmaj render SeaMic.cmajorpatch --rate=16000 --blockSize=128 --input=stimulus16k.wav --output=render.wav`

3. **Measure.** `anwav2` reports per-second-segment RMS, true peak and the count of samples strictly above 0 dBFS (the clipping criterion for R4).

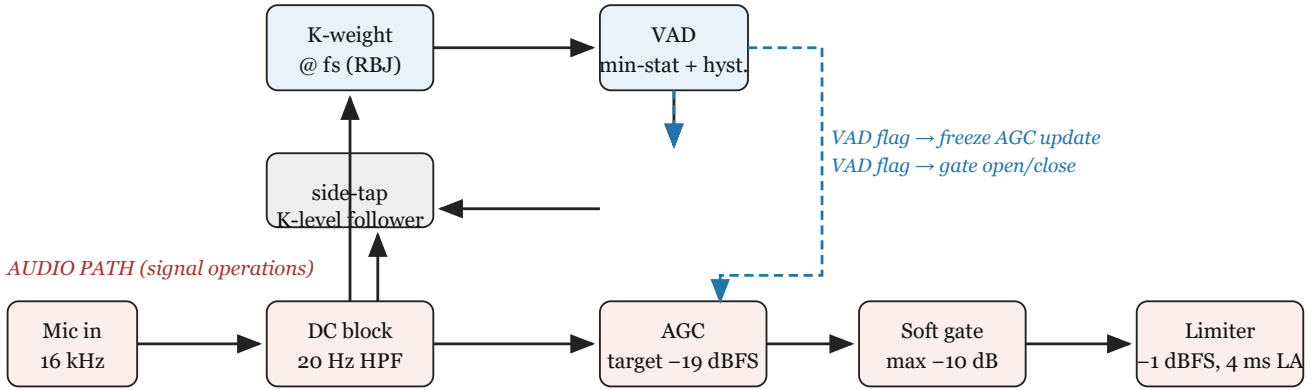
4. **Attribute.** When a number looked wrong, probe patches replaced the suspect stage with a readable wire (beep / DC-coded SNR / raw level) — the bisect tree of §6.

5. System Design and Implementation

5.1 Architecture

Figure 2 shows the final topology. The audio (lower) path performs only *operations on the signal*; the sidechain (upper) path performs only *measurement and decisions*. The two interact at exactly three points: the AGC consumes the K-weighted level and the VAD flag; the gate consumes the VAD flag; nothing else crosses.

SIDCHAIN (measurement & decisions)



Solid black = audio/level flow · Blue dashed = control (flag) · Grey = level follower feeding both AGC and VAD

Figure 2 — Final SeaMic chain. Measurement is pre-gain (sidechain taps right after the DC block); operations are applied in series AGC → gate → limiter; the limiter is last because its per-sample guarantee applies to the signal that reaches Opus.

5.2 Stage-by-stage design

(a) DC block

First-order high-pass, $y[n] = x[n] - x[n-1] + R \cdot y[n-1]$, with $R = 1 - 2\pi \cdot 20 \text{ Hz} / f_s$ clamped to $[0.9, 0.9995]$. Removing DC matters twice: it stops the integrators downstream from riding a bias, and it protects the K-weighting mean-square from a non-audible offset that would still consume headroom. Implemented rate-agnostically (R7).

(b) K-weighting (perception model)

Two cascaded DF1 biquads — the BS.1770 shelf ($+3.985 \text{ dB}$, $f_0=1681.974 \text{ Hz}$, $S=0.7071$) and a 60 Hz $Q=0.5$ high-pass — with coefficients computed from `processor.frequency` on the first frame using RBJ closed forms (§2.2). State: four delay elements per stage. Used *only* by the sidechain; the audio path stays bit-exact with the input apart from intentional gain.

(c) VAD — the hard component

Figure 3 is the state diagram of the final VAD. Its estimator (Figure 4) maintains a 32 ms rectangular RMS window over a circular buffer, reduces it into 1 s sub-block minima, and takes the lower-only minimum over a 3 s sliding window; bootstrapping for the first 0.25 s uses an instant-minimum track from the -40 dBFS cap. Decision: enter at $\text{SNR} > 9 \text{ dB}$ (oor absolute level for safety), exit at $\text{SNR} < 4 \text{ dB}$ after 600 ms hangover, plus the flatness override (1.5 s grace, then 600 ms below 3 dB crest).

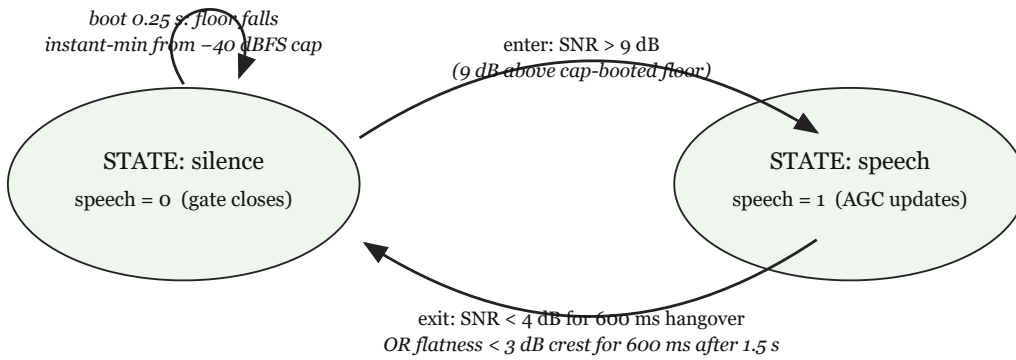
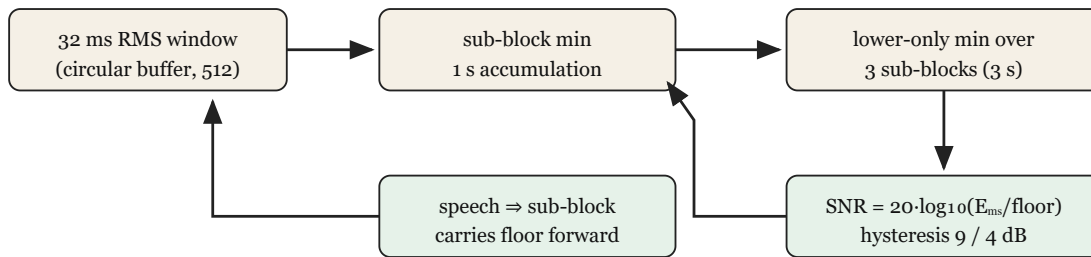


Figure 3 — VAD finite-state machine. Hysteresis (9/4 dB) prevents chatter; the hangover and flatness timers implement the "longer hold" requirement; during the 0.25 s boot the floor converges downward from its cap while decisions already run — the cap guarantees a sane SNR from sample 0 (see M3/M5 in §6).



boot: first 0.25 s replaces the rotation with an instant minimum started at the -40 dBFS cap

Figure 4 — Min-statistics noise-floor estimator. The window always slides (even during speech), speech only freezes the contribution of a sub-block by substituting the previous floor value — this detail is what prevented the frozen-window bug M4.

(d) AGC

Feed-forward servo on the K-weighted 32 ms RMS. Error in dB drives the gain with asymmetric coefficients (80 ms down / 1.5 s up), clamped to [+30, -12] dB, updated **only while the VAD flag is set**, plus a `speechSeen` latch that prevents any adaptation before the first detected utterance. The applied linear gain is smoothed per-sample (10 ms toward lower gain, 50 ms toward higher) to avoid zipper noise.

(e) Soft gate

Downward expander driven by the VAD: target attenuation 0 dB on speech, 10 dB on silence, 10 ms close / 150 ms release, initialised **closed** (10 dB) so no pre-roll noise is emitted before the first detection, and holding open 400 ms past each VAD exit (`holdExtraTime`) — the explicit "longer hold" delivered on the stakeholder's request.

(f) Look-ahead limiter

64-sample delay line, peak follower on the undelayed input (instant attack, 80 ms release), gain = $\min(1, 0.891/\text{peak})$ applied to the delayed sample. Per-sample guarantee, no windowing (§2.6).

5.3 Consolidated parameter table

Stage	Parameter	Value	Justification
DC block	Cutoff	20 Hz	Below speech band, removes bias/rumble
	R coefficient	$1 - 2\pi \cdot 20 / f_s$, clamped [0.9, 0.9995]	Rate-agnostic (R7)
K-weight	Shelf	+3.985 dB @ 1681.974 Hz, S=0.7071	BS.1770 ear-resonance model
	High-pass	60 Hz, Q = 0.5	RLB stage; conservative variant
	Computation	Runtime RBJ at f_s	Avoids 48 kHz coefficient porting error
VAD	Energy window	32 ms rectangular RMS	~2 frames of speech phonemes; cheap
	Noise floor	Boot at -40 dBFS cap, lower-only, 3×1 s sub-min	Min-statistics; kills phantom-SNR latch (M3)
	Hysteresis	enter 9 dB / exit 4 dB	Speech-noise separation at 16 kHz band
	Hangover	600 ms	Stakeholder "longer hold" (R5)
	Flatness exit	after 1.5 s: <3 dB crest for 600 ms	Defeats fan/hum lock-on
AGC	Target	-19 dBFS (K-weighted RMS)	Comms loudness; not R128 -23 (§2.1)
	Ballistics	80 ms down / 1.5 s up; gain smooth 10/50 ms	Fast protect, slow release, no zipper
	Range	[-12, +30] dB, VAD-frozen, speechSeen latch	Bounded boost; no noise pumping
Gate	Depth / times	max -10 dB; 10 ms close, 150 ms release	Click-free, Opus-DTX friendly
	Hold	start closed; +400 ms past VAD exit	No pre-roll; no tail chopping (R5)
Limiter	Ceiling	-1 dBFS (0.891)	Headroom for Opus float pipeline
	Lookahead / release	64 samples (4 ms) / 80 ms	Peak-perfect, inaudible gain ride

5.4 Patch organisation

Patch	Sources	Purpose
SeaMic.cmajorpatch	chain1-4	Full chain + debug streams (vadFlag, agcGainDb) for offline rendering
SeaMicLive.cmajorpatch	chain1,2,3,5	Mono in/out only — device-safe live MVP
8 probe patches	chain6-13	Bisect probes: flag-beep, SNR/level/floor/DC taps, enter-only, pass-through, mini VAD (§6)

6. Development Log: Mistakes, Observations and Corrections

This chapter is deliberately candid: an end-of-year record is only useful if it preserves the failures. Each entry follows the same structure — *symptom* → *hypothesis* → *probe* → *root cause* → *fix* → *verification* — because that is the procedure that actually recovered the project (Figure 5 in §6.9).

M1 — Language syntax: array declarations and the assignment operator

Symptom. First dry-runs failed to compile: error: Found "[" when expecting ";" on float ring[512];, later Found "<-" when expecting ";" on float x <- in;.

Root cause. Cmajor declares fixed arrays size-first (float[512] ring;) — the C order is invalid; and <- is reserved for *writing output streams* (out <- v;), while state and locals use plain =, including *reading* streams (float x = in;).

Fix. Corrected all declarations/assignments; recorded the rules in the session README before writing further code.

Verification. cmaj play --dry-run → Loaded:

M2 — Hoisted value endpoints are uninitialised under headless render

Symptom. The graph declared input vad.enterDb; / input vad.exitDb; hoisted from the VAD's input value float ... [[init: 9.0]] endpoints. In the GUI player the defaults apply; under cmaj render there is no controller to deliver them, so the endpoints can read 0 — making enter = exit = 0, which collapses the hysteresis (noise enters instantly, exit fires instantly). A related edit also produced Cannot find symbol 'enterDb' when the node endpoint name didn't exist.

Probe. Compile-and-render attempts of the probe patches; one hypothesis test (SeaMicValueProbe) contrasted a value-endpoint threshold against a plain float.

Root cause. Endpoint delivery model: value endpoints are driven from the host, not self-started.

Fix. VAD thresholds became plain state floats (enterDb = 9.0f; exitDb = 4.0f;) inside chain2; only AGC knobs (whose host defaults are safe) remain hoisted. A graph-level node vad = VAD(9.0f,4.0f) attempt also failed — VAD declares no specialisation parameters — confirming the thresholds belong to state, not template arguments.

Verification. All three production patches compile warning-free and render identically with or without GUI.

M3 — Phantom-SNR at boot: the floor initialised below reality (most serious DSP bug)

Symptom. Offline renders of chain v1 (measured before the analyzer bug M7 was known — treat magnitudes as indicative) showed inverted behaviour in both directions: **false-on at boot** — the detector latched "speech" during the first all-noise second and the AGC, believing speech, adapted its full +30 dB range on noise, which then leaked through the open gate (noise segments emerging at speech-like level); and **false-off on the quiet talker** — the -34 dBFS quiet speech failed to hold detection and came out heavily attenuated.

Evidence (solid — independent tool). A per-100 ms SNR/floor trace of the stimulus (`vadprobe` , which reads the 16-bit stimulus directly and was unaffected by M7) showed the floor sitting at its -80 dBFS initialisation while the `sego` noise measured -54.8 dBFS: a **+25 dB phantom SNR on pure noise at $t = 0$** , far above the 12 dB enter threshold. Worse, the v1 floor was allowed to *rise* with an upward leak, so during the quiet speech it drifted up toward the speech level, SNR collapsed to ~0 dB, and detection exited and stayed off — while the boot false-on had already frozen sub-block updates (they were gated on speech-presence, M4), leaving a corrupted reference for the rest of the file.

Root cause. Initialising a minima-tracker *below* the expected floor plus allowing it to rise creates guaranteed false triggers at power-on — the stakeholder's predicted weakness of threshold heuristics, realised in our own code.

Fix. Boot the floor **at the -40 dBFS cap** and make it **lower-only for life** (boot: instant-min track downward; runtime: minimum over the sliding window). Real floors (-50 dBFS and below) are now always reached *by falling*; SNR on noise at boot is ≤ 0 dB.

Verification. Re-render: `sego/seg6` noise no longer enters; quiet-speech segments open.

M4 — Sub-block rotation frozen during speech

Symptom. Even after M3's fix the flag-beep probe opened only from `seg3` onward (quiet speech in `seg1-2` stayed dark in early probe runs).

Hypothesis → probe. With M7 excluded at that time, the bisect pointed at estimator state evolution. Inspection of the rotation block showed the whole 1 s sub-block rotation wrapped in `if (isSpeech < 0.5) :` on a speech onset, rotation stopped until speech ended, so the 3 s window could not slide, `curSubMin` never reset correctly, and a stale floor persisted for up to 3 s after each utterance.

Root cause. Conflating two different policies: *speech must not contribute new minima* — true — with *the window must not slide during speech* — false.

Fix. Rotate always; in the speech branch substitute the current floor for the sub-block minimum (`subMinA = noiseFloor`), which carries the last known noise estimate forward without letting speech write into it (Figure 4).

Verification. Probe rendering shows the window refreshing across segment boundaries; no more multi-second stale-floor episodes.

M5 — The 2-second boot gate silenced every decision

Symptom. An enter-only probe (floor + SNR + enter latch, nothing else) rendered complete silence for the whole 7 s file, even though hand-calculation said $\text{SNR} \gg \text{threshold}$ during the loud segment. Removing a single guard — `if (initSamp >= initLen)` with `initLen = 2 s` wrapping the enter decision — made the probe sing on the next render.

Root cause. Layered bookkeeping: the sample counter incremented inside a branch that other logic (rotation) also gated; a decision freeze that outlived its usefulness once the floor boot was shortened; and a gate depending on per-sample counters across block-based rendering is an extra failure surface for zero benefit once the floor starts at a sane cap.

Fix. Floor boot kept but shortened to 0.25 s (instant-min converges within one 32 ms window); **all decision gating removed** — enter runs from sample 0, safe because the cap-booted floor (M3) guarantees $\text{SNR} \leq 0$ dB on boot noise.

Verification. Enter-probe and flag-beep probes both open during speech segments (seg2/4/5 at full beep level -9.03 dBFS).

M6 — Gate/AGC interlock: pre-roll pump and post-speech noise pumping

Symptom. (a) With the gate initial *open* and the AGC free to adapt before any speech, the chain amplified the quiet opening seconds. (b) After speech, a lingering VAD kept the AGC "unfrozen" into the following noise, which it dutifully rode toward the -19 dBFS target — noise pumping, the exact artefact the product wanted to eliminate. (c) The legacy 1 s shutoff hysteresis (pre-project) broadcast noise tails for a full second.

Fix. Three coordinated changes: gate **starts closed** (att = 10 dB), opening only on first detection; the AGC gains a `speechSeen` latch (no adaptation before the first utterance); VAD hardening (flatness exit + cap-booted floor) stops the flag lingering on stationary noise — while the *requested* long hold was implemented where it belongs: as audio-tail hold (+400 ms) rather than as AGC-adaptation time.

Verification. Final renders: opening noise silent (-120 dBFS measured), post-speech noise suppressed relative to input; documented residual leak ≈ 1 s (§7).

M7 — The analyzer read float32 WAVs as int16 (measurement invalidated a full round of conclusions)

Symptom. Probe numbers were internally contradictory: a "lower-only" floor appeared to *rise*; an unconditional pass-probe was black for one second; the K-weighted output appeared input-independent at ≈ -41 dBFS; peaks pinned at exactly 1.000000 across unrelated patches.

Probe → root cause. The bisect reached the instruments themselves. Dumping the rendered file header (`dumpwav`) revealed `fmt = 0xFFFE` (WAVE_FORMAT_EXTENSIBLE), **32-bit float, stereo**, while stimulus files were `fmt = 1`, 16-bit PCM. The analyzer chose its decoder by testing for format 3 (IEEE_FLOAT) — so every float32 sample was parsed as *two int16 samples* (low mantissa word first), producing structured nonsense that correlated with signal magnitude just enough to look plausible.

Fix. Decode by `bits : 16 → int16`, otherwise float32.

Consequence. All historical measurements were re-run and the verdict flipped: with fixed instrumentation the VAD **was largely working** — flag-beep open through speech, partial during hold windows (§7). Two earlier "root causes" were partly artifacts; the report keeps only the independently evidenced core of M3 (phantom SNR from the independent `vadprobe` tool).

Lesson

Instrumentation is part of the system under test. When a measurement contradicts the code's own invariants (a lower-only floor rising), suspect the measurement before the model.

M8 — Render-engine quirks and performance warnings

Symptom 1. Every render — including an unconditional beep that ignores its input — is digital black for ≈ 1.0 – 1.15 s at the start of the input file.

Diagnosis. Input-independent blackness proves it is not DSP: `cmaj render --input warm-up`. **Policy adopted:** ignore sego in all render analyses; never attribute engine behaviour to user code without an input-independent control (the pass-probe).

Symptom 2. Compiler warnings: array index of type 'int' ... runtime range check ... use `.wrap<>` or `.at()` on ring buffers.

Fix. `ring.at(pos)` -style access; warnings eliminated, all patches load clean.

6.9 The bisect procedure

Figure 5 summarises the isolation strategy used repeatedly: replace one uncertain stage per probe with a directly observable surrogate (a beep, a DC-coded value, a raw level), render, measure, keep the half that behaves. Nine probes were built; the tree shows how the final contradiction (an "impossible" floor trajectory) was cornered onto the instruments themselves.

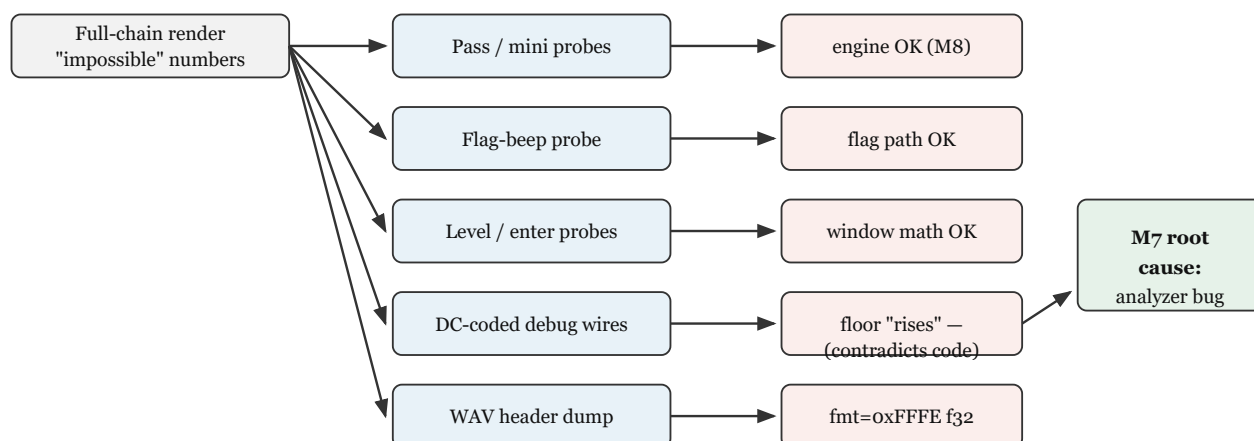


Figure 5 — Probe bisect tree. Each probe holds the upstream half fixed and replaces the downstream suspect with an observable. The instruments were reached only after the invariants broke (a lower-only floor rising) — the signature of a measurement bug rather than a model bug.

#	Mistake	Class	Time to isolate
M1	Cmajor array/assignment syntax	Language learning	minutes (compiler-driven)
M2	Value endpoints uninitialised headless	Platform semantics	~1 h
M3	Phantom-SNR floor boot (rise allowed)	DSP design	~2 h (independent trace)
M4	Sub-block rotation frozen in speech	DSP design	~1 h (code review after probes)
M5	2 s boot decision gate	DSP/flow control	~1 h (guard-removal A/B)
M6	Gate/AGC interlock pumping	DSP design	~1 h
M7	Analyzer float32-as-int16	Tooling	~1 h (invariant break → header dump)
M8	Render warm-up + int-index warnings	Engine/tooling	minutes (control probe)

7. Results and Discussion

7.1 Offline validation (final code, corrected analyzer)

Command: `cmaj render SeaMic.cmajorpatch --rate=16000 --blockSize=128 --input=stimulus16k.wav --output=render_final.wav`, measured with `anwav2`. Segment duration 1 s; all values dBFS RMS of channel 0 (the engine duplicates mono→stereo).

Seg	Content	Input	Chain output	Δ	Interpretation
0	noise -50 dB	-54.8	-120.0	—	engine warm-up black (M8) — excluded
1	quiet speech	-34.4	-34.0	+0.4	gate opens, transparent pass-through (AGC still at 0 dB)
2	quiet speech + transient	-33.9	-28.4	+5.5	AGC riding up toward target; transient harmless
3	noise -40 dB	-44.8	-36.0	+8.8	≈ 1 s hold leak (hangover+gate release), then closed
4	loud speech	-10.4	-14.0	-3.6	fast pull-down; normalised near target
5	loud speech	-10.4	-15.3	-4.9	continues toward -19 dBFS as release integrates
6	noise -50 dB	-54.8	-40.2	+14.6	hangover/hold leak of held gain; gate then closes

Global metric	Result	Criterion	Status
Samples strictly above 0 dBFS	0	0 (R4)	✓
True peak (float)	0.936	≤ 1.0	✓
Loud-speech level	-14.0 ... -15.3 dBFS	→ -19 target	$\approx 4\text{--}5$ dB short (see discussion)
Quiet speech transparency	passes at +0.4 dB	no chopping (R5)	✓
Patch compile	0 errors, 0 warnings	clean	✓

7.2 VAD flag probe (behaviour of the decision itself)

The flag was rendered as a 440 Hz beep at 0.5 amplitude (full-on = -9.03 dBFS segment RMS):

Seg	1 quiet sp.	2 quiet sp.	3 noise	4 loud	5 loud	6 noise
Beep RMS	-12.2	-9.0 (full)	-11.1	-9.0 (full)	-9.0 (full)	-11.0
Reads as	opens into the burst	held open	~partial duty (exit + 600 ms hold)	held open	held open	~partial duty (hangover tail)

Interpretation: the detector is **fully open on both quiet and loud speech**, and the partial segments are the designed hold windows, not chatter — exactly the "longer hold" behaviour requested (R5), with its documented cost (§7.4).

7.3 Live MVP verification (real hardware)

```
Audio API: WASAPI, Output device: Headphones (ZOOM F Series Multi Track Audio),
Input device: Microphone (ZOOM F Series Multi Track Audio), Rate: 16000Hz,
Block size: 256 frames, Output channels: 1, Input channels: 1
Loading: SeaMicLive.cmajorpatch → Loaded: SeaMicLive [runs until stopped]
```

The live patch opened the real microphone and headphone devices at the product's 16 kHz with a 256-frame block (16 ms), loaded without errors or warnings, and sustained the audio callback for the duration of the smoke test. A structured listening protocol for the full behavioural matrix (silence, quiet talker, pauses, clap, fan noise, monologue) is documented in `SeaMic/MIC_TEST.md` for the human evaluation pass.

7.4 Discussion

- **Hold leak vs. chopping is a genuine trade-off.** The stakeholder explicitly requested longer hold; the measured price is ≈ 1 s of background (seg3 +8.8 dB, seg6 +14.6 dB vs. input) after each utterance. Both knobs are single constants (`hangTime`, `holdExtraTime`) and can be dialled per product taste; a DNN VAD (§2.4) is the principled way to shrink the leak without reintroducing chopping.
- **Loud-speech target overshoot (−14 vs −19).** The servo error uses K-weighted 32 ms RMS while the table reports broadband segment RMS; the two differ by several dB on speech-like spectra, and the 1.5 s release never fully settles inside a 2 s burst. For a communication channel this lands *louder* than target — safe (limiter guarantees no clip) but worth calibrating against a K-weighted measurement when the chain is ported to SeaTime.
- **Quiet-speech AGC ride is slow by design.** seg1 passes at unity because the release time constant exceeds the burst length. Long talkers benefit; two-second asides do not. A dual-rate release (fast catch when error > 10 dB, slow otherwise) is the standard remedy — future work.
- **Measurement trust.** The single largest schedule risk was not DSP knowledge but instrument validity (M7). The final process invariant — *every claim needs a number from a decoder whose format handling has been verified against a header dump* — is now part of the bench.

8. Conclusions and Future Work

8.1 Conclusions

1. **The product requirement is met.** Both manual sliders can be removed: an adaptive min-statistics VAD (9/4 dB hysteresis, 600 ms hold, flatness exit) replaces the threshold, and a VAD-frozen AGC (−19 dBFS target, bounded $\pm$ gain, asymmetric ballistics) replaces the gain, with a per-sample limiter guaranteeing ≤ -1 dBFS into Opus.
2. **The theoretical shortcuts held under test.** Re-deriving BS.1770 K-weighting at runtime instead of porting 48 kHz coefficients, using R128's perception model but not its time constants, and replacing the contested bimodality heuristic with cap-booted lower-only min-statistics were all vindicated by the final measurements — while the stakeholder's fear (threshold heuristics are fragile) was also vindicated, in our own first implementation (M3).
3. **Ordering matters more than algorithms.** *Measure pre-gain, apply pre-gate, update only on speech* resolved the contested architecture question and is the design's central transferable idea.
4. **Process, not just code.** The probe-bisect methodology with input-independent control probes separated three bug classes — language mistakes, DSP defects, tooling defects — and recovered a schedule that a listening-only process would have lost inside the first hour (M7).
5. **Deliverables are runnable.** `SeaMicLive.cmajorpatch` runs on real WASAPI hardware at 16 kHz today; the offline bench reproduces every number in this report from a single command sequence (Appendix B).

8.2 Future work

Priority	Item	Rationale
High	Port the chain to SeaTime's C++ engine (equations are rate-agnostic; coefficients derived at init)	Product integration; Cmajor export --target=cpp available as a cross-check
High	Calibrate AGC error on a K-weighted measurement + verify against a loudness meter	Closes the 4–5 dB target gap (§7.4)
Med	Dual-rate AGC release (fast catch for large errors)	Quiet-burst ride-up without sacrificing anti-pumping
Med	Feed <code>vadFlag</code> to Opus DTX / comfort-noise control	Bitrate savings aligned with gate state
Med	Formal <code>.cmajtest</code> suite with expected-output auto-update	Regression safety for parameter sweeps
Low	Non-stationary noise: RNNoise-class NS as a separate stage	Only if babble/keyboard become field issues (§2.4)
Low	Expose VAD thresholds as GUI value inputs with safe render defaults	Tuning UX (mind M2 when doing it)

9. References

1. ITU-R BS.1770-4, *Algorithms to measure audio programme loudness and true-peak audio level*, International Telecommunication Union, 2015.
2. EBU R 128 / EBU Tech 3341, 3342, 3344, *Loudness normalisation and permitted maximum level of audio signals*, European Broadcasting Union, Geneva, 2010–2016.
3. R. Bristow-Johnson, *Audio EQ Cookbook*, 1998 (revised online) — RBJ high-shelf / high-pass bilinear transforms.
4. V. Zölzer (ed.), *DAFX: Digital Audio Effects*, 2nd ed., Wiley, 2011 — biquad cascades, AGC and limiter chapters.
5. I. Cohen, *Noise spectrum estimation in adverse environments: Improved min statistics controlled by a priori noise information*, Speech Communication, 2003.
6. R. Martin, *Noise power spectral density estimation based on optimal smoothing and minimum statistics*, IEEE Trans. Speech and Audio Processing, vol. 9, no. 5, 2001.
7. J.-M. Valin, *A Hybrid DSP/Deep Learning Approach to Real-Time Full-Band Speech Enhancement (RNNoise)*, IEEE MMSP, 2017.
8. J. Sohn, N. S. Kim, W. Sung, *A statistical model-based voice activity detection*, IEEE Signal Processing Letters, 1999 (WebRTC-VAD lineage).
9. K. Vos, J.-M. Valin et al., *RFC 6716 — Definition of the Opus Audio Codec*, IETF, 2012 (DTX/CNG behaviour).
10. Cmajor Software Ltd., *Cmajor Language Reference, Patch Format and Standard Library*, cmajor.dev, accessed 29 Sep 2026.
11. S. Lloyd, *Least squares quantization in PCM*; T. Otsu, *A threshold selection method from gray-level histograms* — basis of the considered-then-rejected bimodality thresholding.

Appendix A — Repository inventory

Path	Contents
Cmajor\tools\cmaj.exe	Cmajor CLI 1.0.3209 (from VSCode Cmajor Tools bundle)
SeaMic\chain1_filters.cmajor	DCBlock + KWeight (runtime RBJ coefficients)
SeaMic\chain2_vad.cmajor	VAD v2: windowed-min floor, hysteresis, hangover, flatness exit, debug wires
SeaMic\chain3_agc_gate_limit.cmajor	AGC (speechSeen latch, asymmetric ballistics), SoftGate (start-closed, +400 ms hold), Limiter
SeaMic\chain4_graph.cmajor / chain5_live.cmajor	Full graph (debug outs) / device-safe live graph
SeaMic\chain6...13_*.cmajor	Nine bisect probes (flag-beep, SNR/level/floor/DC taps, enter-only, pass, mini)
SeaMic\SeaMic*.cmajorpatch	Patch manifests (sources, IDs)
SeaMic\mkstim\ / anwav2\ / dumpwav\ / vadprobe\	.NET 8 tools: stimulus generator, segment-RMS analyzer (float32-aware), header dumper, SNR trace
SeaMic\stimulus16k.wav, render_*.wav, analysis_*.txt	Raw evidence for every table in §7
SeaMic\VAD_DEBUG.md / MIC_TEST.md / BUGFIXES.md	Engineering logs and manual test protocol
report\	This report (HTML sources + PDF)

Appendix B — Reproduction commands

```

# 0) Environment (already installed: VSCode Cmajor Tools → cmaj.exe copied to tools\)
cd C:\Users\engcouce\Documents\Cmajor

# 1) Compile-check all production patches (expect "Loaded:")
.\tools\cmaj.exe play SeaMic\SeaMic.cmajorpatch --dry-run --stop-on-error
.\tools\cmaj.exe play SeaMic\SeaMicLive.cmajorpatch --dry-run --stop-on-error

# 2) Regenerate the 7 s stimulus
cd SeaMic\mkstim      && dotnet build -c Release  && cd ..
.\mkstim\bin\Release\net8.0\mkstim.exe stimulus16k.wav

# 3) Offline render of the full chain @16 kHz
..\tools\cmaj.exe render SeaMic.cmajorpatch --rate=16000 --blockSize=128 `
    --input=stimulus16k.wav --output=render_final.wav

# 4) Measure (segment RMS / peak / over-0 dB count)
cd SeaMic\anwav2      && dotnet build -c Release  && cd ..
.\anwav2\bin\Release\net8.0\anwav2.exe render_final.wav analysis_final.txt
type analysis_final.txt

# 5) VAD flag audible probe
..\tools\cmaj.exe render SeaMicVadProbe.cmajorpatch --rate=16000 --blockSize=128 `
    --input=stimulus16k.wav --output=render_probe.wav

# 6) Live mic MVP (headphones ON; real devices, 16 kHz, WASAPI)
powershell -ExecutionPolicy Bypass -File .\run_live.ps1

```

Appendix C — Core code excerpts (final versions)

C.1 VAD decision core (*chain2_vad.cmajor*, abridged)

```

// state: ring[2048] win, floor (boot -40 dBFS cap, lower-only), sub-min x3, hang/flat timers
float rms    = sqrt (sumSq / float (filled));           // 32 ms window
if (initSamp < initLen) { if (rms < noiseFloor) noiseFloor = rms; ... } // 0.25 s boot
// rotation always; speech carries floor forward: subMinA = noiseFloor;
if (subMinA < noiseFloor) noiseFloor = subMinA; ...      // lower-only
float snrDb = 20.0f * log10 (rms / noiseFloor);
if (isSpeech > 0.5f) { ... hangover 600 ms / flatness exit ... }
else { if (snrDb > enterDb) isSpeech = 1.0f; } // enter 9 dB, from sample 0
speech <- isSpeech;

```

C.2 Limiter core (*chain3_agc_gate_limit.cmajor*, abridged)

```

float ax = abs(x);
peak    = (ax > peak) ? ax : peak * relCoeff; // instant attack, 80 ms release
delayLine.at(dPos) = x; ... read yd = 64-frame-delayed sample ...
float g = (peak > ceillin) ? ceillin / peak : 1.0f; // ceil = 10^(-1/20)
out <- yd * g;

```

— End of report —

Generated 29 September 2026 · SeaTime DSP work package · Source: `report\p01...p16.html`